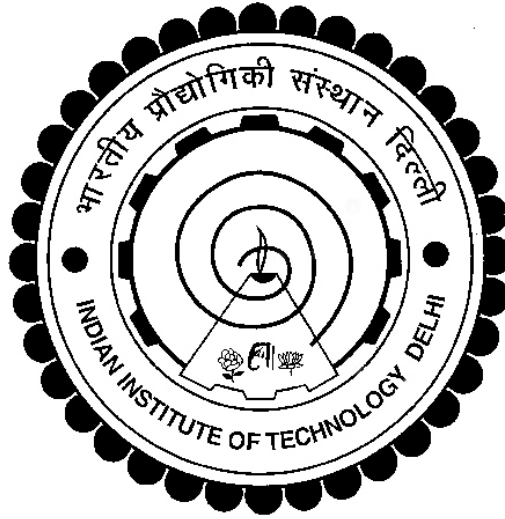




Department of Physics, IIT Delhi

Course Code : PYD562
Academic Year : 2025 -2026, Semester - IV

Machine Learning Based Continuous Variable Quantum Key Distribution

Nitish (2024PHS7145)
Shadab Anjum (2024PHS7155)

Supervisor:- Prof. Neel Kanth Kundu
Co-Supervisor:- Prof. Bhaskar Kanseri

Signature of Nitish:

Signature of Shadab:

Signature of Supervisor: 

Certificate

This is to certify that the master thesis entitled **Machine Learning Based Post Processing of Continuous Variable Quantum Key Distribution**, submitted by Nitish Kathuria (2024PHS7145) and Shadab Anjum (2024PHS7155), to the Indian Institute of Technology, Delhi, for the award of the degree of Master of Science, is a bona fide record of the research work done by them under my supervision.

Prof. Neel Kanth Kundu
CARE, IIT Delhi



Signature of the thesis supervisor

Statement of Originality

We hereby confirm that we have written this report on our own and that the scientific and technical content is original and not copied from any online or offline source.

Nitish Kathuria (2024PHS7145)
Shadab Anjum (2024PHS7155)

Department of Physics, IIT Delhi

Abstract

Continuous Variable Quantum Key Distribution (CV-QKD) offers information-theoretic security using standard telecom components, with the GG02 protocol as its foundation. GG02 employs Gaussian-modulated coherent states and homodyne detection, but practical limits arise from fiber loss, excess noise, and reconciliation inefficiency.

This report presents the complete evolution of our CV-QKD simulation framework across two semesters. Starting from a faithful GG02 baseline, we first introduce a post-selection enhancement: a Gaussian probabilistic filter at Alice’s side and a non-Gaussian notch filter at Bob’s side, which dynamically optimize modulation and emulate a higher-quality channel — purely through software, without any hardware changes. We replace LDPC-based error correction with polar codes to substantially reduce reconciliation latency, and integrate a rigorous finite-size composable security analysis. Finally, we adapt the Isolation Forest machine learning algorithm as a sifting mechanism to detect and remove anomalous noisy states before parameter estimation, significantly boosting secret key rates in high-noise finite-size regimes. Results across SNR and channel-loss scenarios demonstrate that combining polar codes with ML-based state filtering provides a computationally efficient, software-only pathway to practical high-performance CV-QKD.

Low-Density Parity-Check (LDPC) codes are commonly used for error correction in CV-QKD but suffer from high computational complexity and latency due to iterative decoding. In this work, we replace the conventional LDPC decoder with a trained neural network model that learns the decoding process directly from noisy channel data. The network is trained over a range of signal-to-noise ratio (SNR) values to ensure robustness under varying channel conditions. Simulation results show that the neural network achieves comparable and, in several cases, improved performance in terms of decoding accuracy and bit error rate. Additionally, it significantly reduces decoding time by eliminating iterative message-passing. This demonstrates the potential of neural network-based decoding as an efficient and scalable alternative for practical CV-QKD systems

Contents

1	Introduction and Motivation	5
2	Theoretical Background: The GG02 Protocol	5
2.1	State Preparation and Channel Transmission	6
2.2	Parameter Estimation	6
2.3	Mutual Information and Holevo Bound	7
2.4	Asymptotic and Finite-Size Secret Key Rate	8
2.5	Results	8
3	Post-Selection Enhancement via Dual Filters	10
3.1	Protocol Overview	11
3.2	Role Of Alice Gaussian Filter	12
3.3	Role of Bob's Non-Gaussian notch Filter	12
3.4	Asymptotic Key Rate After Applying Alice Filter	13
4	Error Correction Framework	14
4.1	Data Normalization	14
4.2	Discretization and Bit Mapping	14
4.3	Polar Encoding	15
4.4	Successive Cancellation Decoding of Polar Codes	15
4.4.1	Initialization	15
4.4.2	Recursive Decoding Functions	15
4.4.3	Hard Decision and Frozen Bits	15
4.4.4	Partial Sum Update	16
4.5	Verification	16
4.6	Results	16
5	Isolation Forest: ML-Based State Sifting	18
5.1	Algorithm Description	18
5.2	Effect on Excess Noise and SKR	20
5.3	Results	21
6	LDPC Decoding using Neural Network	22
6.1	System Structure	22
6.2	LDPC Encoder	23
6.3	LDPC Decoder	25
6.3.1	Min-Sum Iterative Decoding Algorithm	25
6.4	Result	27
7	Conclusions	28
8	Future Work	28
	Acknowledgements	29
	References	29

1 Introduction and Motivation

Quantum Key Distribution (QKD) offers a fundamentally secure method for distributing secret keys by leveraging the principles of quantum mechanics rather than relying on computational hardness assumptions. Unlike classical cryptographic protocols, which are vulnerable to attacks by future quantum computers, QKD ensures security even against adversaries with unlimited computational power, grounded in the no-cloning theorem and the measurement-disturbance trade-off.

Early protocols such as BB84 used discrete variables (single photons), but practical implementation challenges have driven significant interest toward Continuous Variable QKD. CV-QKD encodes information in the quadrature amplitudes of coherent states and employs homodyne or heterodyne detection, allowing the use of standard telecommunication components and achieving high key rates over optical fibers.

Among the various CV-QKD protocols, the Gaussian-modulated coherent-state protocol (GG02) stands out for its strong security proofs and robustness under realistic conditions. Its compatibility with existing optical networks makes CV-QKD, particularly GG02, a promising candidate for scalable, real-world quantum communication in the era of emerging quantum technologies.

Nevertheless, GG02’s practical performance is fundamentally limited by fixed modulation variance, channel loss, and the need for accurate parameter estimation — especially in rapidly fluctuating channels. This motivates exploring a purely software-driven approach that leverages post-selection to virtually manipulate the effective modulation and channel behaviour without touching the optical setup.

Furthermore, reconciliation efficiency — the ability to correct errors between Alice and Bob’s correlated data — remains a critical bottleneck. We address this by replacing LDPC decoding with polar codes, which offer lower latency and enable large-block experiments. To raise the secret key rate further, we integrate the Isolation Forest algorithm for ML-based sifting of anomalous states.

Our ultimate motivation is to push CV-QKD beyond conventional limits by uniting quantum communication theory with modern AI, opening a new pathway toward smarter, faster, and more resilient quantum-secure networks.

2 Theoretical Background: The GG02 Protocol

The security of QKD arises from the fundamental laws of quantum mechanics. The CV-QKD pipeline comprises several stages: state preparation and transmission, parameter estimation, error correction, privacy amplification, and verification.

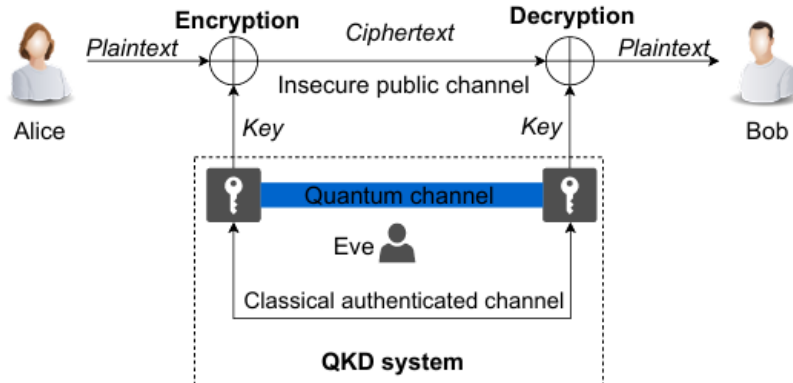


Figure 1: GG02 Protocol Overview

2.1 State Preparation and Channel Transmission

Alice prepares coherent states $|\alpha\rangle$ with Gaussian-modulated quadratures x and p :

$$\alpha = x + ip, \quad x, p \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

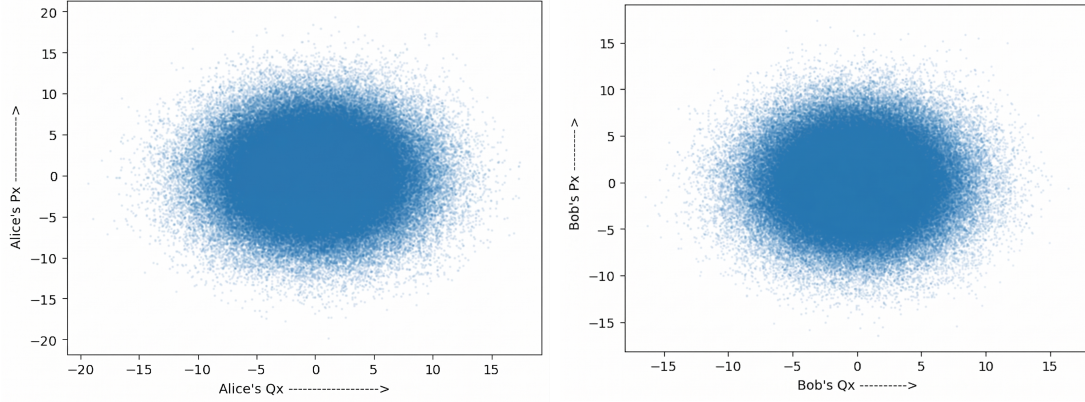
where σ^2 is the modulation variance. These coherent states are transmitted through an optical fiber of length L with transmissivity

$$T = 10^{-\alpha_{\text{loss}} L/10}, \quad (2)$$

where α_{loss} is the fiber attenuation coefficient in dB/km. Assuming Eve has full control over the quantum channel, Bob measures a randomly chosen quadrature (x or p) for each state, yielding:

$$y = \sqrt{T\eta}x + z, \quad z \sim \mathcal{N}(0, 1 + T\eta\xi + v_{\text{el}}), \quad (3)$$

where η is Bob's detector efficiency, ξ is the excess noise referred to the channel input, and v_{el} is the electronic noise variance.



The distribution shows the original distribution reflects how Alice's transmitted Gaussian-modulated coherent states are affected by the quantum channel.

2.2 Parameter Estimation

Alice and Bob estimate the quantum channel characteristics from a subset of their shared data using the empirical covariance matrix. The estimated transmissivity and excess noise are:

$$\hat{T} = \frac{1}{\eta \sigma_x^4} \hat{C}_{xy}^2, \quad \hat{\xi} = \hat{\sigma}_z^2 - v_{\text{el}} - 1, \quad (4)$$

$$\hat{\sigma}_z^2 = \frac{1}{m} \sum_{i=1}^m \left(y_i - \sqrt{\eta T} x_i \right)^2. \quad (5)$$

The signal-to-noise ratio (SNR) at Bob's detector is:

$$= \frac{TV_s}{T(V_0 - 1) + 1 + \xi}, \quad (6)$$

where V_s is the modulation (signal) variance and V_0 is the shot-noise variance.

2.3 Mutual Information and Holevo Bound

Mutual information I_{xy} measured the amount of shared Knowledge between Alice and Bob, it tells us how well Bob's measurement outcome reveals Alice Signals despite noise. The key rate is obtained by subtracting the Holevo bound from the mutual information.

$$I_{xy} = \frac{1}{2} \log_2(1 + \text{SNR}) \quad (7)$$

where $\text{SNR} = \text{signal to noise Ratio}$. To compute the **Eve's Holevo information**,

$$\chi_{BE} = S(V_E) - S(V_{E|B}), \quad (8)$$

which upper bounds the information accessible to Eve under collective Gaussian attacks. The reduced covariance matrix of Eve's modes is given by

$$V_E = \begin{pmatrix} \omega I & \psi Z \\ \psi Z & \phi I \end{pmatrix}, \quad (9)$$

where

$$\psi := \sqrt{T(\omega^2 - 1)}, \quad \phi := T\omega + (1 - T)\mu. \quad (10)$$

Here, $I = \text{diag}(1, 1)$ and $Z = \text{diag}(1, -1)$.

The conditional covariance matrix after Bob's homodyne measurement is

$$V_{E|B} = V_E - b^{-1} \begin{pmatrix} \gamma^2 \Pi & \gamma \theta \Pi \\ \gamma \theta \Pi & \theta^2 \Pi \end{pmatrix}, \quad (11)$$

where

$$b := T\eta(\mu + \xi) + 1 - T\eta + v_{el}, \quad (12)$$

$$\Pi = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad (13)$$

and

$$\gamma := \sqrt{\eta(1 - T)(\omega^2 - 1)}, \quad \theta := \sqrt{\eta T(1 - T)(\omega - \mu)}. \quad (14)$$

The thermal-noise parameter is

$$\omega = \frac{T\xi - T + 1}{1 - T}. \quad (15)$$

For Gaussian states, the von Neumann entropy S depends only on the symplectic eigenvalues $\{\nu_i\}$ of the corresponding covariance matrix:

$$S = \sum_i h(\nu_i). \quad (16)$$

For a two-mode covariance matrix written in block form

$$V = \begin{pmatrix} A & C \\ C^T & B \end{pmatrix}, \quad (17)$$

the symplectic eigenvalues are given by

$$\nu_{\pm} = \sqrt{\frac{\Delta \pm \sqrt{\Delta^2 - 4 \det V}}{2}}, \quad (18)$$

where

$$\Delta = \det A + \det B + 2 \det C. \quad (19)$$

The entropy function for a symplectic eigenvalue ν is given by

$$h(\nu) = \frac{\nu + 1}{2} \log_2 \left(\frac{\nu + 1}{2} \right) - \frac{\nu - 1}{2} \log_2 \left(\frac{\nu - 1}{2} \right). \quad (20)$$

Using the above procedure, we can find the symplectic eigenvalues of V_E and $V_{E|B}$ and hence their von Neumann entropies and consequently the Holevo Bound.

2.4 Asymptotic and Finite-Size Secret Key Rate

The maximum secure key rate achievable per signal when the number of transmitted signals tends to infinity. So, statistical fluctuations Vanishes. The key rate is obtained by subtracting the Holevo bound from the mutual information.

$$K_{\text{asym}} = \max(0, \beta I_{AB} - S_E).$$

where $\beta \in (0, 1]$ is the reconciliation efficiency, K = Asymptotic Key Rates, χ_E = Holevo Bound on Eve Information and β = reconciliation efficiency

For finite keys we include reconciliation efficiency β , the key fraction n/N , and others statistical Fluctuations terms, penalty for leakage is also included. The finite-size secret-key rate used in the report is

$$K_{\text{comp}} = \frac{n}{N} [\beta I_{AB} - \chi_{BE} - D_n] \quad (21)$$

The finite-size composable security correction term is given by

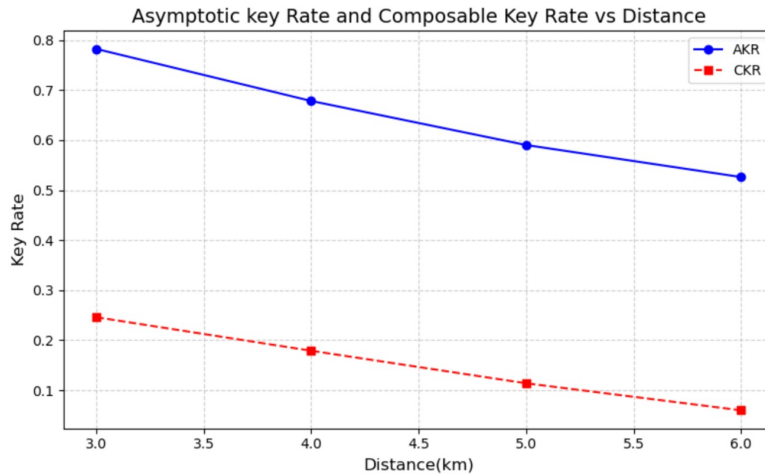
$$D_n = D_1 + D_2 \quad (22)$$

where

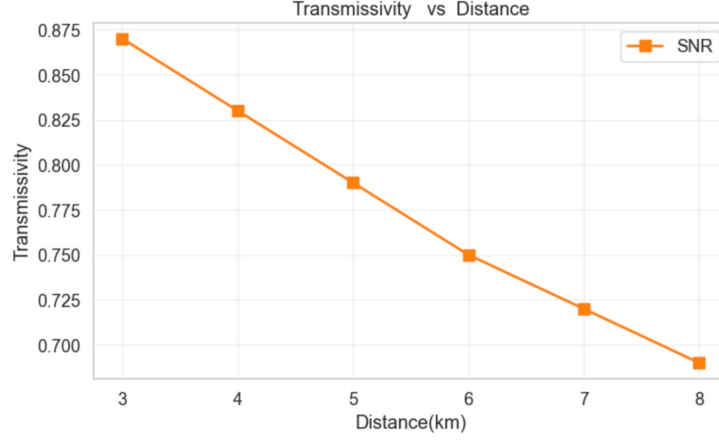
$$D_1 = (2H + 3) \sqrt{\frac{\log_2 \left(\frac{2}{\epsilon_1} \right)}{n}}, \quad D_2 = \frac{2}{n} \log_2 \left(\frac{1}{\epsilon_2} \right). \quad (23)$$

Here, n is the number of signals used for key generation, H is an upper bound on the Shannon entropy of the discretized variable, and ϵ_1, ϵ_2 are composable security parameters.

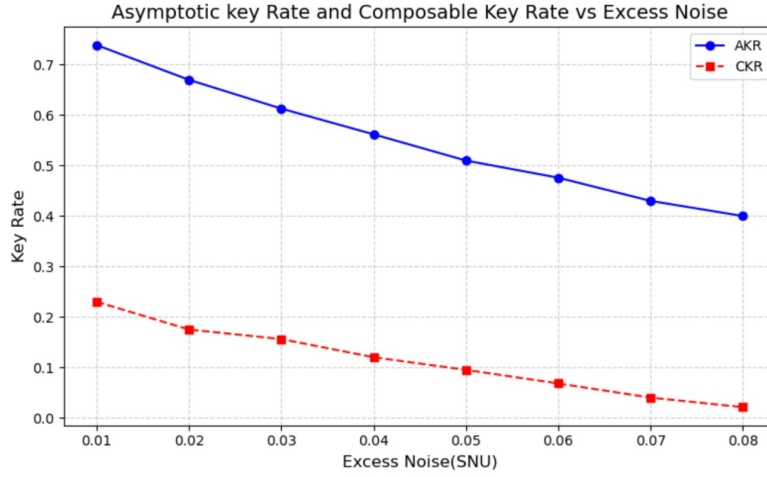
2.5 Results



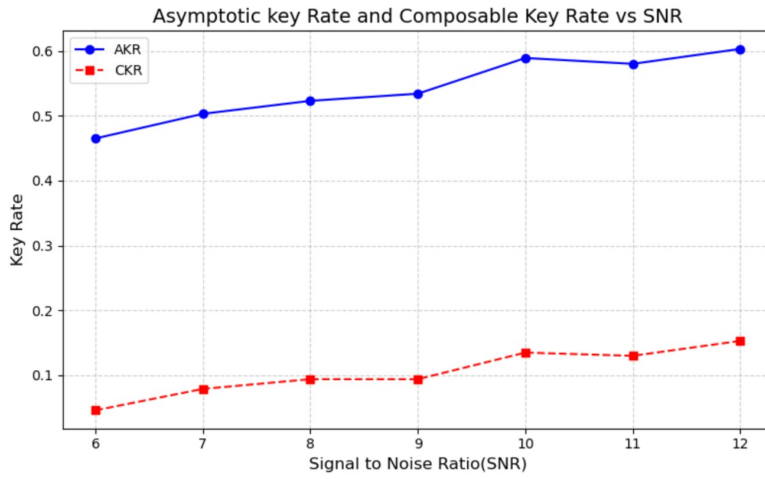
- Figure shows the variation of asymptotic and composable key rates with transmission distance. Both rates decrease as the distance increases, since higher fiber loss reduces the correlation between Alice and Bob and makes the effect of noise more significant. The composable key rate remains lower than the asymptotic key rate because it accounts for finite-size security effects.



- Figure shows the variation of transmissivity with transmission distance. As the distance increases, the transmissivity decreases due to attenuation in the optical fiber caused by absorption and scattering losses. This reduction in transmissivity leads to weaker received signals at Bob's side, which affects the overall performance of the CV-QKD system



- Figure shows the variation of asymptotic and composable key rates with excess noise (in shot-noise units). Both key rates decrease as excess noise increases, since it reduces the correlation between Alice and Bob and degrades the effective signal quality. The composable key rate remains lower due to finite-size security constraints.



- Figure shows the variation of asymptotic and composable key rates with signal-to-noise ratio (SNR). As SNR increases, the channel becomes cleaner and the correlation between Alice and Bob improves, leading to a higher secret key rate. The composable key rate remains lower than the asymptotic key rate because it includes finite-size security effects.

3 Post-Selection Enhancement via Dual Filters

We can increase the performance of CV-QKD without adding any new hardware by introducing intelligent software-level post-processing. By applying a Gaussian filter at Alice's side and a non-Gaussian filter at Bob's side, we virtually optimise the modulation and emulate a better channel, strengthening Alice–Bob correlations while reducing Eve's information. This software-only approach enables significant key-rate improvements, demonstrating that enhanced performance is achievable purely through digital processing.

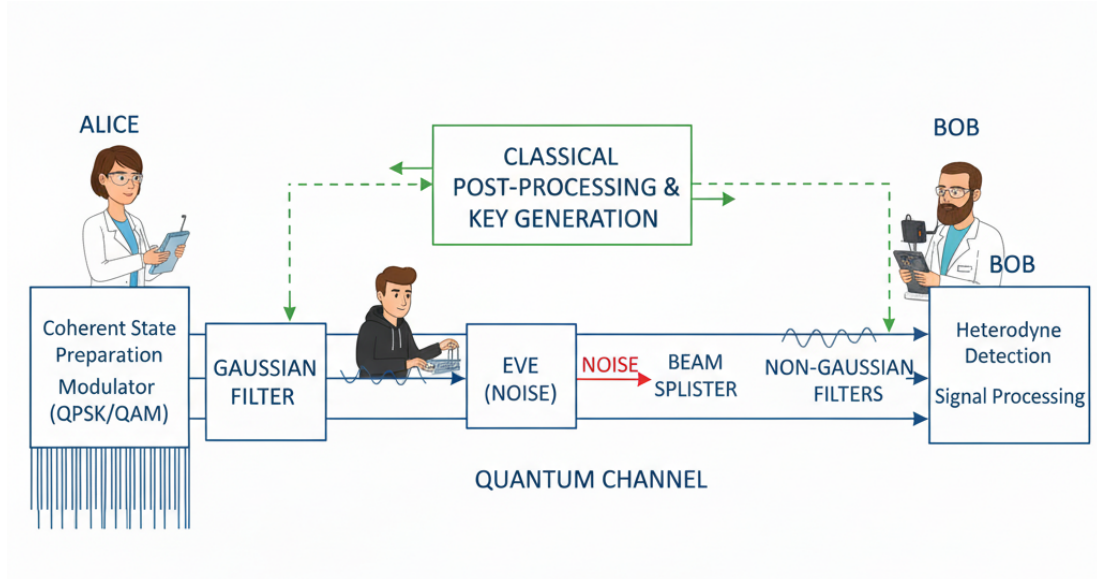
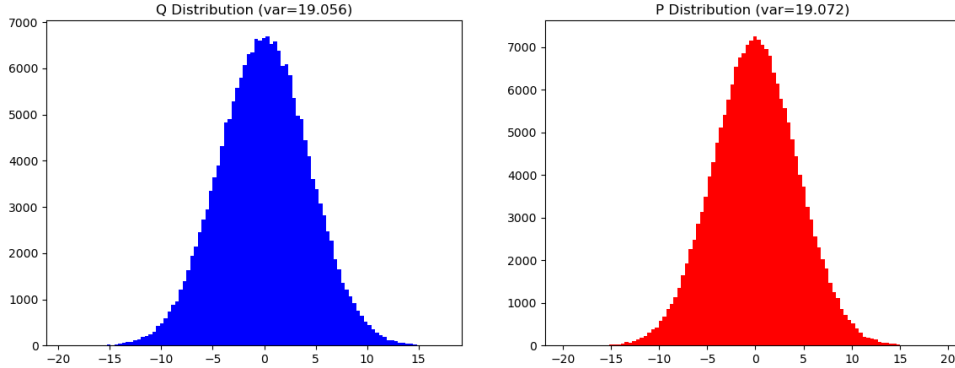


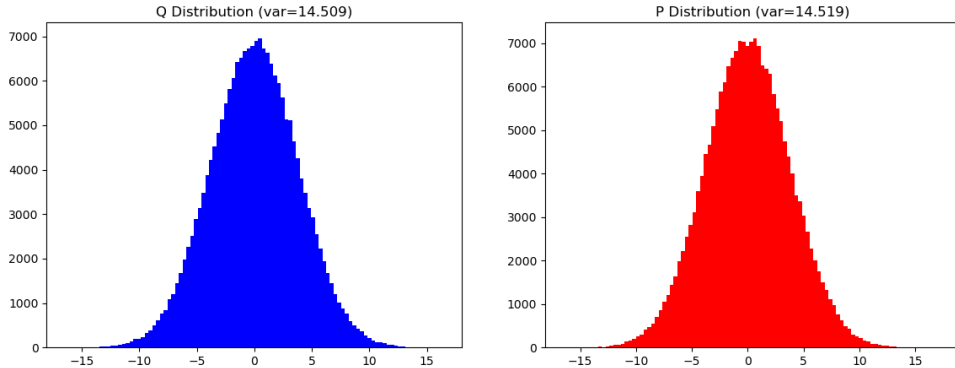
Figure 2: GG02 Protocol Setup with filters

3.1 Protocol Overview

- Alice prepares and records modulation values x_q and p_a for each pulse and sends the optical pulses. Bob measures and records x_b and p_b . Both keep full time-stamped lists.
- Alice applies her Gaussian probabilistic filter to her local record and decides (keep/discard) for each index. Bob applies his non-Gaussian cut to his measured list and decides keep/discard for each index. The filters are local operations on classical data.
- After filtering each side has a keep/discard decision per index. They announce (over the authenticated classical channel) which indices survived. Then they take the intersection: only the pulses that both kept are kept for key generation and parameter estimation.
- On the surviving subset they estimate channel parameters, correct errors, and apply privacy amplification.
- **The filters do not physically change photons in flight** — they are post-selection on the classical data. This makes experiments much easier: only software changes.
- Alice applies her Gaussian post-selection after the states have been prepared and sent and after Bob has taken his measurements.



- The above Figure shows, Histogram of Alice's raw Q and P quadrature values showing the original Gaussian modulation with variances around 19. These distributions represent the coherent states prepared by Alice before applying any post-selection filter



- The last two Figures shows, Histogram of Bob's measured Q and P quadrature values before any post-selection. Due to channel loss and noise introduced during transmission, the variances reduce to approximately 14.5, showing how the quantum channel degrades the signal even before applying Bob's filter.

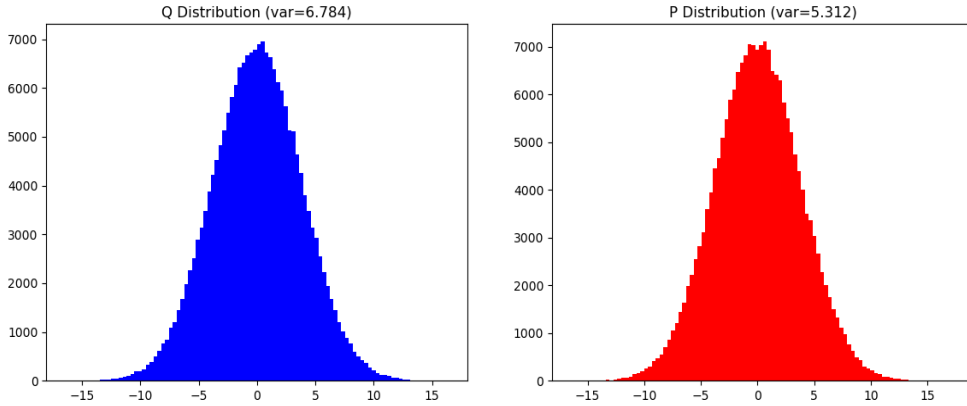
3.2 Role Of Alice Gaussian Filter

- The Alice filter is a smooth acceptance probability applied to her classical modulation values. A common functional form:

$$P(Q_a) \propto \exp(-g_q^2 Q_a^2)$$

and similarly we can apply filter to the P quadrature of alice and the gain we will take for P quadrature will be g_p

- Because the filter is Gaussian and smooth, the kept ensemble remains close to Gaussian. That makes security analysis still approximately valid and easier.
- **Alice's filter = variance tuning, not random hiding.** Alice's Gaussian filter is equivalent to probabilistically reducing the modulation variance of the states she effectively uses.
- GG02 has an optimal modulation variance for each channel loss/noise; if Alice's physical modulation was too large, post-selection lets her keep only the samples that effectively make the modulation behave like the optimal variance.
- That increases I_{AB} (mutual information) relative to the unfiltered case.
- So, we implemented the Alce gaussian filter and Applied the Filter to the Original States Prepared by the Alice to check how it affects the original states.



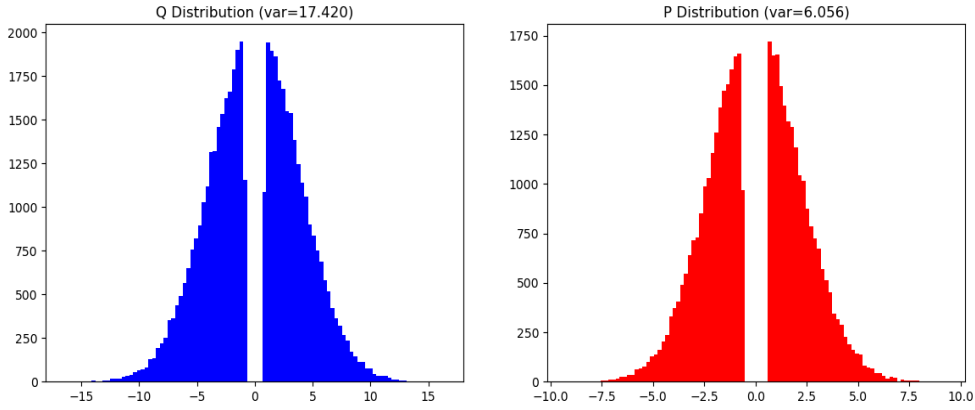
- Alice's Gaussian filtering narrows the variance without changing the Gaussian shape, increasing the per-surviving-sample mutual information because Bob's noise becomes small relative to the reduced modulation spread—leading to higher key rates.

3.3 Role of Bob's Non-Gaussian notch Filter

- A non-Gaussian filter is typically an indicator (hard) rule rather than a smooth weight. Typically a notch-like filter used at the side of Bob, that practically choose the measurements that belongs to $|Q_x| > c$ and discards others.

$$F_B(x_b) = \begin{cases} 0, & -c_x < x_b < c_x, \\ 1, & \text{elsewhere.} \end{cases}$$

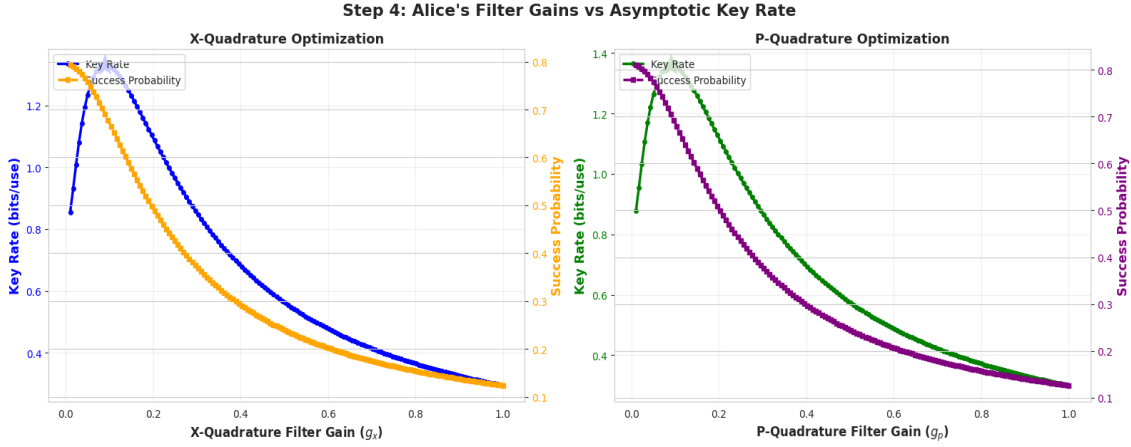
- That non-Gaussian selection preferentially keeps outcomes that are better correlated with Alice’s record or that correspond to higher effective signal.
- This can emulate an improved channel and so lift the key rates above unfiltered GG02.
- A non-Gaussian filter produces highly non-Gaussian output statistics, and such selections can do things a Gaussian filter can’t, for example, preferentially remove the most noisy central outcomes.
- This can increase the conditional correlations for the surviving data in a way that a Gaussian attenuation alone cannot.
- Bob’s non-Gaussian choice is deliberately more aggressive statistically; it helps but forces you to do heavier security calculations.
- We have therefore implemented the Bob’s Filter Using Python and then Studied the distribution of remaining data after Applying the filter.



- Bob’s non-Gaussian filter removes low-amplitude (low SNR) points, creating the central gap and keeping only strongly correlated samples—effectively emulating a higher-quality channel and boosting the key rate.

3.4 Asymptotic Key Rate After Applying Alice Filter

- Alice’s Gaussian filter preserves the Gaussian nature of the transmitted states, meaning the covariance structure stays intact, and the security analysis can continue to rely on standard Gaussian formulas.
- As the filter gain increases, the key rate initially rises because the reduced variance better matches the channel condition and reduces Eve’s information.



- The success probability decreases monotonically with gain, reflecting the tradeoff that aggressive filtering keeps fewer samples but with higher per-symbol correlation.
- The key-rate curve therefore shows a clear optimum, where the modulation variance and the success probability balance to give the maximal asymptotic key rate.

4 Error Correction Framework

The error correction pipeline consists of four primary stages: data normalization, discretization, Polar encoding, and Successive Cancellation (SC) decoding. This process ensures that continuous source data is robustly transmitted over a noisy channel.

4.1 Data Normalization

Before discretization, the raw input data $X = \{x_1, x_2, \dots, x_N\}$ is normalized to achieve numerical stability and a standard dynamic range. We apply Z-score normalization:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (24)$$

The normalized value $\tilde{x}_i$ is then computed as:

$$\tilde{x}_i = \frac{x_i - \mu}{\sigma} \quad (25)$$

4.2 Discretization and Bit Mapping

To interface with binary error correction codes, the normalized continuous values $\tilde{x}_i$ are converted into bits.

1. **Quantization:** A k -bit uniform quantizer maps $\tilde{x}_i$ to one of 2^k discrete levels.
2. **Bit Conversion:** Each level is mapped to a binary vector $\mathbf{b} \in \{0, 1\}^k$. The concatenation of these vectors forms the information sequence $\mathbf{u}_A$.

4.3 Polar Encoding

Polar codes exploit the phenomenon of channel polarization. The information bits $\mathbf{u}_{\mathcal{A}}$ are combined with frozen bits $\mathbf{u}_{\mathcal{A}^c}$ (typically set to 0) to form the vector $\mathbf{u}$. The codeword $\mathbf{x}$ is generated using the generator matrix G_N :

$$\mathbf{x} = \mathbf{u} \cdot G_N, \quad G_N = B_N F^{\otimes n} \quad (26)$$

where $F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ and B_N is the bit-reversal permutation matrix.

4.4 Successive Cancellation Decoding of Polar Codes

The Successive Cancellation (SC) decoding algorithm is a greedy search through the code tree of a Polar code. For a code of length $N = 2^n$, the decoder processes the received LLRs (Log-Likelihood Ratios) to estimate the original bit sequence $\hat{u}_1^N$.

4.4.1 Initialization

The input to the decoder is the vector of LLRs from the channel, $L_i^{(1)}$, defined as:

$$L_i^{(1)} = \ln \left(\frac{P(y_i | x_i = 0)}{P(y_i | x_i = 1)} \right), \quad i \in \{1, \dots, N\} \quad (27)$$

4.4.2 Recursive Decoding Functions

The SC decoder relies on two primary recursive operations, often referred to as the f and g functions. Let a and b be two incoming LLRs from a parent node.

The Left-Branch Function (f) The LLR for the left child is calculated using the f function, which does not depend on previous bit decisions:

$$f(a, b) = 2 \operatorname{arctanh} \left(\tanh \frac{a}{2} \tanh \frac{b}{2} \right) \quad (28)$$

In practical hardware implementations, the min-sum approximation is often used:

$$f(a, b) \approx \operatorname{sgn}(a) \operatorname{sgn}(b) \min(|a|, |b|) \quad (29)$$

The Right-Branch Function (g) The LLR for the right child is calculated using the g function. This calculation is conditional on the hard-decision bit $\hat{u}_{sum}$ returned from the left sub-tree:

$$g(a, b, \hat{u}_{sum}) = b + (-1)^{\hat{u}_{sum}} a \quad (30)$$

4.4.3 Hard Decision and Frozen Bits

Once the recursion reaches the leaf nodes (the n -th level), a hard decision is made for each bit u_i . Let $\mathcal{A}$ be the set of information bit indices and $\mathcal{A}^c$ be the set of frozen bit indices:

$$\hat{u}_i = \begin{cases} 0 & \text{if } i \in \mathcal{A}^c \\ 0 & \text{if } i \in \mathcal{A} \text{ and } L_i^{(n)} \geq 0 \\ 1 & \text{if } i \in \mathcal{A} \text{ and } L_i^{(n)} < 0 \end{cases} \quad (31)$$

4.4.4 Partial Sum Update

After a leaf node decision, the bits must be propagated back up the tree. For a node at level t with length M , the partial sums are updated as:

$$\hat{v}_{parent,[i]} = \hat{v}_{left,[i]} \oplus \hat{v}_{right,[i]} \quad (32)$$

$$\hat{v}_{parent,[i+M/2]} = \hat{v}_{right,[i]} \quad (33)$$

where $i \in \{1, \dots, M/2\}$ and $\oplus$ denotes XOR addition.

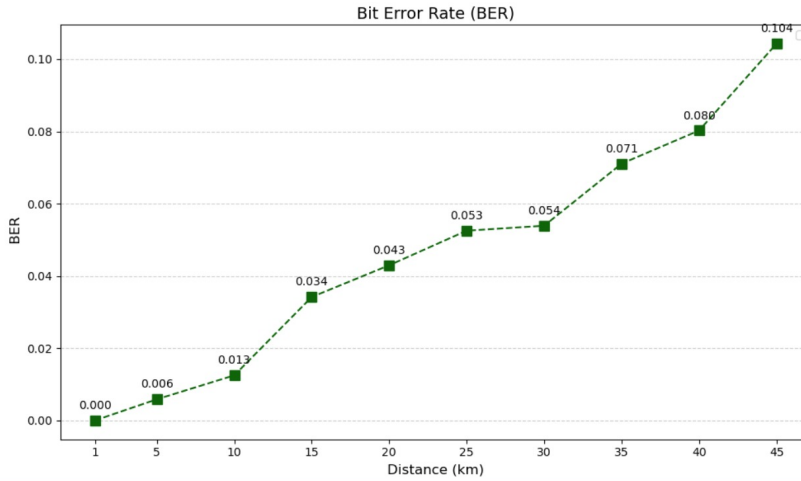
- We define the variables Bit Error Rate and Frame Error Rate for the error correction module as:

$$\begin{aligned} \text{BER} &= \frac{\text{Total number of bit errors}}{\text{Total number of bits transmitted}} \\ \text{FER} &= \frac{\text{Number of frames where } \hat{u}_1^N \neq u_1^N}{\text{Total number of transmitted frames}} \end{aligned} \quad (34)$$

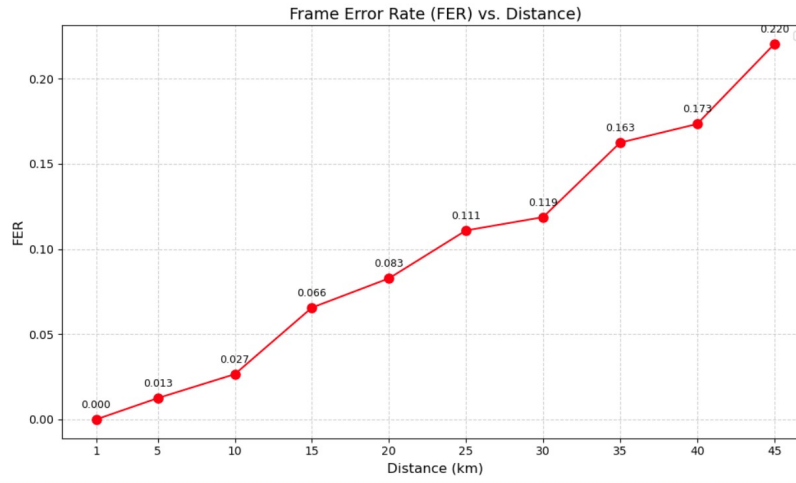
4.5 Verification

After error correction, Bob computes a hash of his corrected string and discloses it to Alice. If the hashes match, the verification stage is successful and the strings are appended together for privacy amplification. If not, the block is discarded.

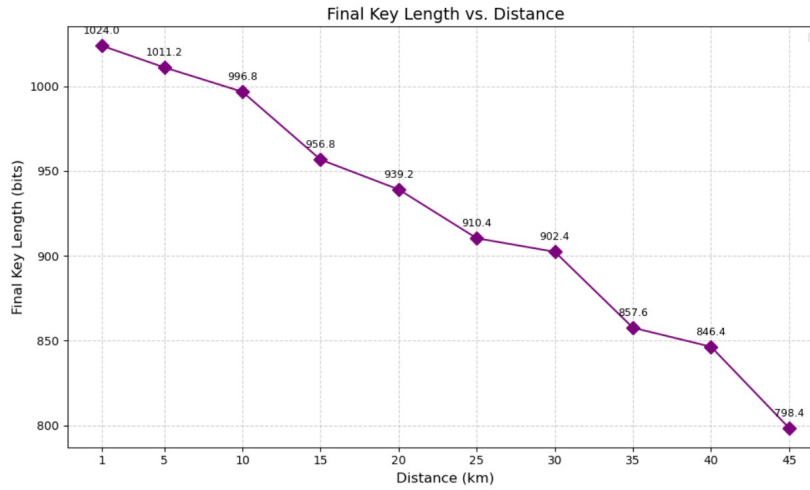
4.6 Results



- Figure shows that the bit error rate (BER) increases steadily with transmission distance. This occurs because longer fiber lengths introduce greater attenuation and noise, which degrade the received signal and reduce decoding accuracy. The rising BER therefore indicates lower channel reliability and more severe signal distortion at larger distances.



- Figure shows that the frame error rate (FER) increases with transmission distance. As the distance grows, optical loss and channel noise become more significant, causing more frames to be decoded incorrectly. The rise in FER from 0 at 1 km to 0.22 at 45 km indicates a clear degradation in communication reliability over longer links.



- Figure shows that the final key length decreases monotonically with transmission distance, dropping from 1024 bits at 1 km to 798.4 bits at 45 km. This reduction is caused by increasing channel loss and noise at larger distances, which degrade the correlation between Alice and Bob and increase reconciliation and privacy-amplification overhead. As a result, fewer bits remain available for the final secure key.

5 Isolation Forest: ML-Based State Sifting

- Isolation Forest is an unsupervised anomaly-detection algorithm that isolates outliers by repeatedly partitioning the data with random splits; anomalies require fewer splits and therefore have shorter average path lengths in the forest, which yields a high anomaly score.
- It identifies and discards miniblocks (or samples) whose features deviate strongly from the typical channel behavior, treating them as corrupted or glitchy.
- By removing these anomalous blocks before parameter estimation, the statistical estimate of noise excludes rare high-noise events, lowering the measured excess noise.
- Reducing excess noise decreases the variance of the received signal that is not explained by the transmitted modulation, which raises the signal-to-noise ratio (SNR) used for parameter estimation and reconciliation.
- In Gaussian-modulated CV-QKD, the received variable follows

$$y = \sqrt{T}x + z.$$

Dividing by x (for non-zero x) gives

$$\frac{y}{x} = \sqrt{T} + \frac{z}{x},$$

which separates the deterministic channel parameter $\sqrt{T}$ from the noise contribution. In the absence of significant noise, $\frac{y}{x}$ remains approximately constant around $\sqrt{T}$, independent of x . Therefore, using $(x, y/x)$ as features helps normalize the data and makes deviations due to noise more explicitly visible.

- In this transformed feature space, normal channel samples form a tight cluster around $\frac{y}{x} \approx \sqrt{T}$, while noisy or corrupted samples exhibit large fluctuations due to the term $\frac{z}{x}$, especially for small x . The Isolation Forest algorithm exploits this structure by assigning higher anomaly scores to points that deviate significantly from the clustered region. These high-score points correspond to noise-dominated or unreliable states and can be removed during sifting to improve the overall quality of the key generation process.

5.1 Algorithm Description

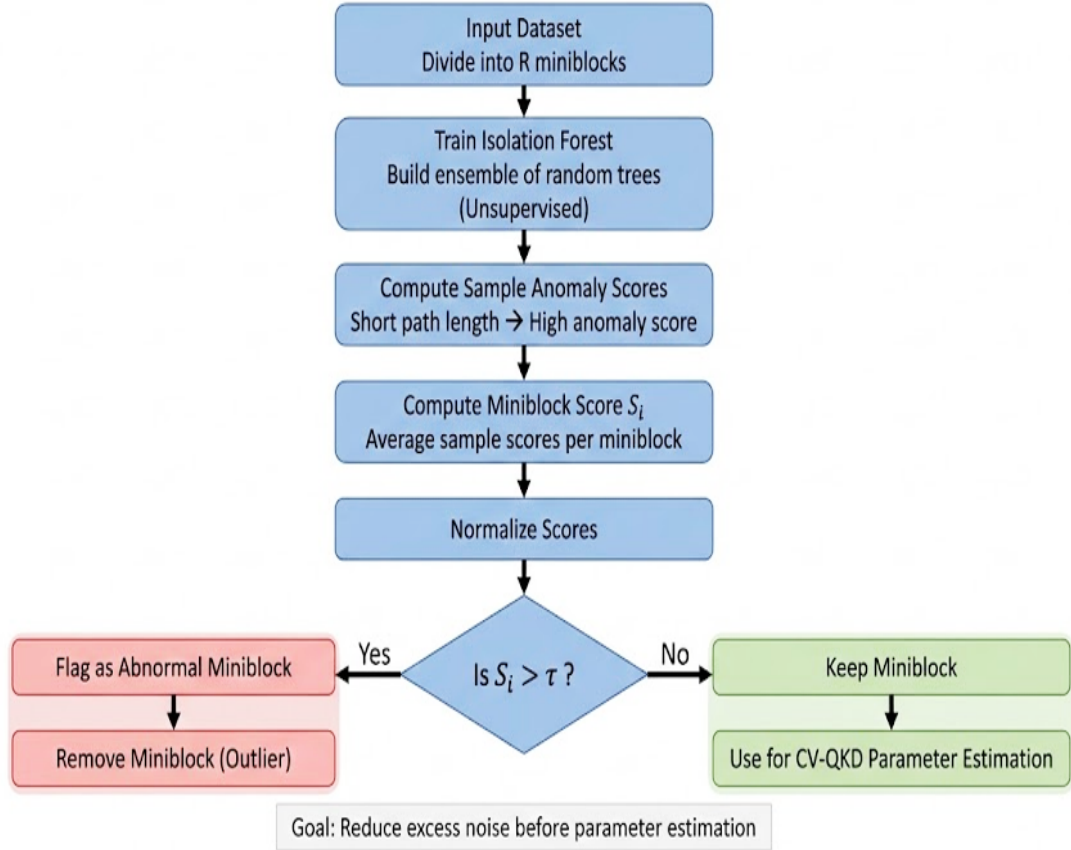
It isolates outliers by repeatedly partitioning the data with random binary splits: anomalies require *fewer* splits than normal points and therefore have shorter average path lengths in the forest. A high anomaly score indicates a likely outlier.

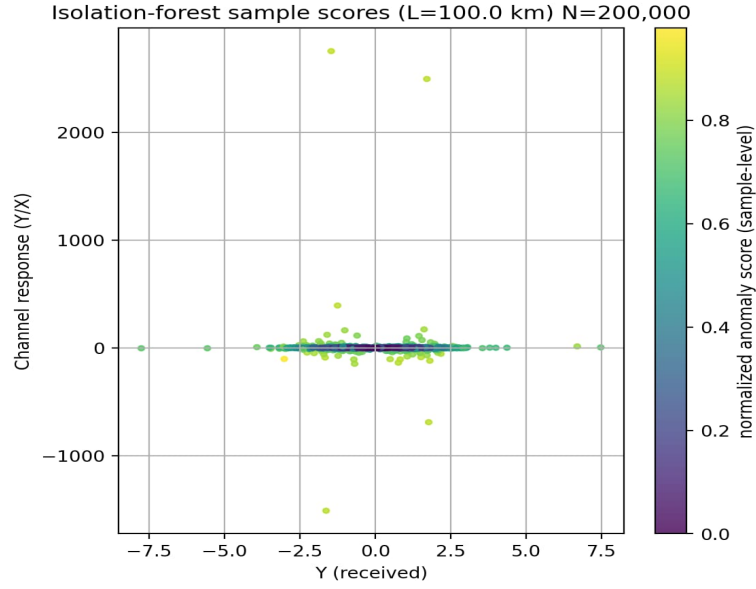
The dataset $\mathcal{M} = \{B_1, B_2, \dots, B_R\}$ is divided into R miniblocks, each containing k samples. The algorithm scores each miniblock and removes those whose normalised score exceeds a threshold τ .

Algorithm 1 Isolation-Forest-Based Sifting Algorithm

```
1: Input:  
2:  $M = \{B_1, B_2, \dots, B_R\}$  // Dataset divided into  $R$  miniblocks  
3:  $B_i = \{m_1, m_2, \dots, m_k\}$  // Each miniblock contains  $k$  samples  
4:  $Q$  // Number of isolation trees  
5:  $\tau$  // Outlier threshold  
6: Output: Outliers (Abnormal miniblocks)  
7: Initialize score set  $S \leftarrow \emptyset$   
8: Initialize Outliers  $\leftarrow \emptyset$   
9: Build isolation forest:  $Iforest \leftarrow \text{Build\_IForest}(B_1, Q)$   
10: for  $i = 1$  to  $R$  do  
11:    $Result \leftarrow \text{Score}(Iforest, B_i)$   
12:    $S_i \leftarrow \text{Mean}(Result)$   
13: end for  
14: Normalize scores:  $S \leftarrow \frac{S}{\max(S)}$   
15: Outliers  $\leftarrow \{B_i \mid S_i > \tau\}$   
16: return Outliers
```

Isolation Forest Based Sifting – Flowchart





- Figure illustrates the anomaly scores obtained using the Isolation Forest algorithm for received CV-QKD samples at a transmission distance of 100 km. The densely clustered points near the origin correspond to normal channel behavior with low anomaly scores, indicating high-confidence valid states. In contrast, sparsely distributed peripheral points exhibit higher anomaly scores and are identified as anomalous or noise-dominated states, which can be removed during the sifting process to improve the quality of the key generation

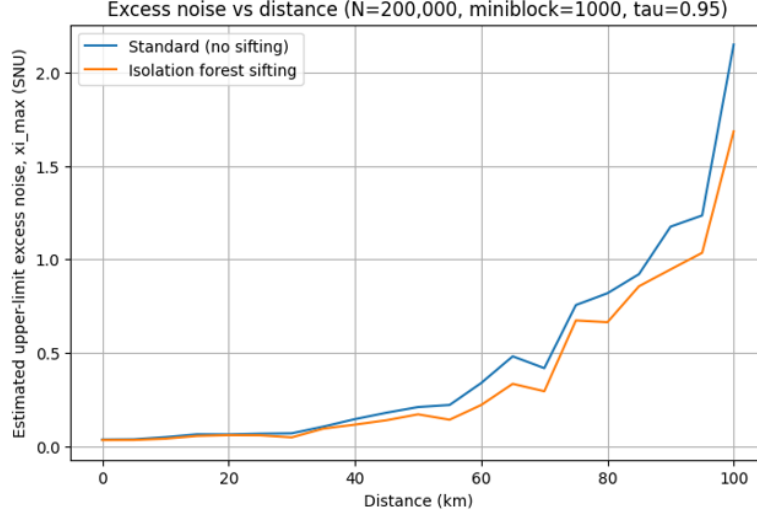
5.2 Effect on Excess Noise and SKR

By removing anomalous miniblocks before parameter estimation, the statistical estimate of noise excludes rare high-noise events. This:

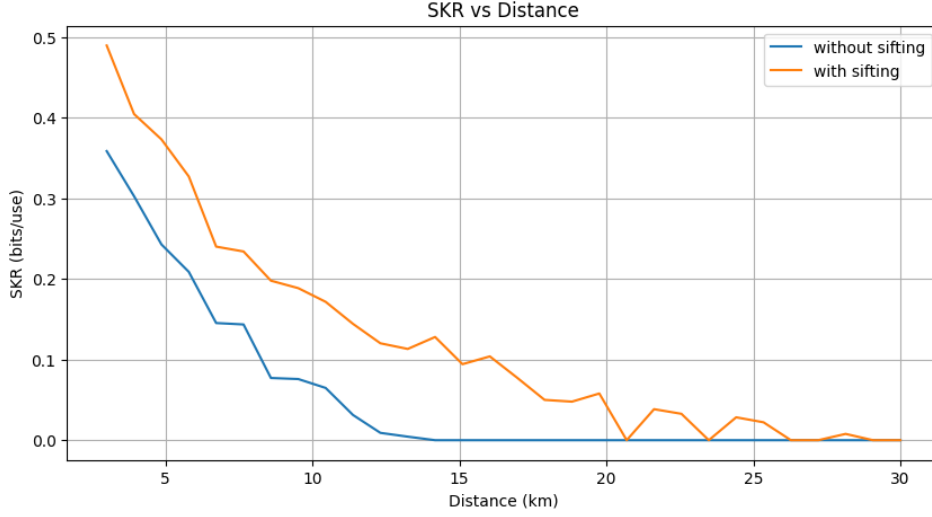
- Reduces the estimated excess noise $\hat{\xi}$,
- Raises the SNR used in parameter estimation and reconciliation,
- Boosts the secret key rate — most significantly in high-noise or disturbed channel conditions.

Under low-noise conditions the sifted and unsifted performances converge, demonstrating robustness without degrading ideal-channel performance.

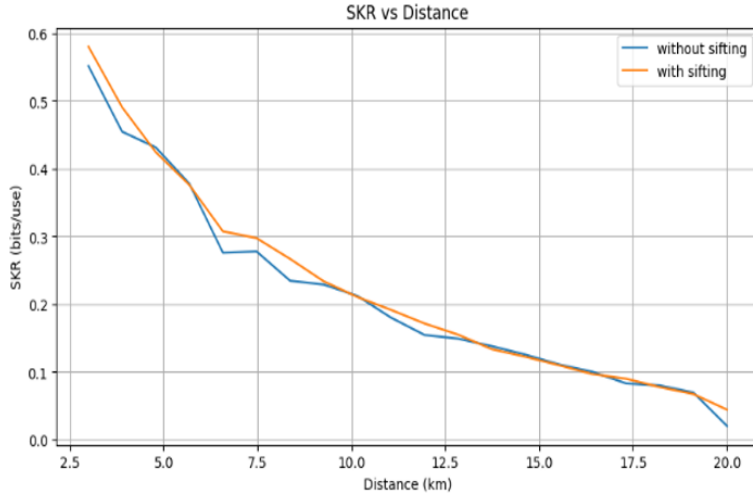
5.3 Results



- Figure shows that excess noise increases with transmission distance due to accumulated channel loss and noise. The Isolation Forest-based sifting consistently results in lower estimated excess noise by removing anomalous samples. This leads to more accurate parameter estimation and improved system performance.



- The SKR decreases with distance for both cases, but Isolation Forest-based sifting provides a noticeable improvement at shorter distances where noise is more significant. This is because anomalous noisy blocks are removed before key generation. At larger distances, the two curves nearly converge as channel loss becomes the dominant factor.



- Figure compares the secret key rate (SKR) with and without Isolation Forest-based sifting. In the low-noise regime, both curves remain very close, showing that the algorithm does not significantly alter the key rate when the channel is already clean. This indicates that Isolation Forest mainly acts as a selective filter for noisy or anomalous samples and becomes more beneficial in regimes where the channel noise is higher.

6 LDPC Decoding using Neural Network

With the development of machine learning and deep learning in recent years, applications have become more extensive, and even the field of communications has been greatly affected. In the process of communication, errors often occur due to noise interference. Therefore, in this work, we combine machine learning and low-density parity-check (LDPC) codes, and use machine learning to reduce decoding errors.

The method employed here is *Belief Propagation*, also known as sum-product message passing. The algorithm chosen is the **min-sum algorithm**, which uses machine learning to train each signal's own weight. After comparing with the original method (fixed weight), it can reduce the error effectively after machine learning.

6.1 System Structure

Low-Density Parity-Check (LDPC) codes are linear block codes widely used for error correction due to their excellent performance near the Shannon limit and efficient iterative decoding. In our CV-QKD reconciliation framework, LDPC coding is used to correct mismatches between Alice's transmitted bits and Bob's received bits in the presence of channel noise. The overall communication pipeline is illustrated in Figure 4. A message signal is first passed through a *Channel Encoder* that applies LDPC coding, producing an LDPC-encoded signal. The encoder maps a binary information vector into a longer coded sequence that satisfies the parity constraints defined by the parity-check matrix. This encoded signal is then transmitted over the channel, where it is subjected to additive noise. The received (noisy) signal is subsequently fed into a *Channel Decoder* that recovers the original message as the LDPC-decoded signal.

System Structure

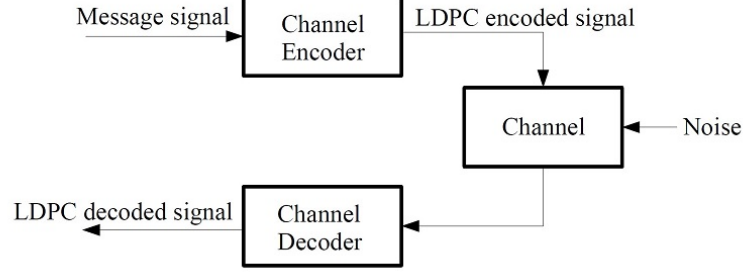


Figure 3: System structure of LDPC-based communication with a neural-network-aided decoder.

6.2 LDPC Encoder

- The encoding process multiplies the message vector $\mathbf{u}$ with a generator matrix G to produce the codeword $\mathbf{x}$:

$$\mathbf{u}G = [u_1, u_2, \dots, u_{(j-i)}] \begin{bmatrix} G_{11} & \cdots & G_{1j} \\ \vdots & \ddots & \vdots \\ G_{(j-i)1} & \cdots & G_{(j-i)j} \end{bmatrix} = \mathbf{c} = [x_0, x_1, \dots, x_j]. \quad (35)$$

The original message has length $(j-i)$ bits; after multiplication by G , we obtain a j -bit encoded codeword.

For decoding convenience, the binary message is transformed by mapping “0” $\rightarrow$ “+1” and “1” $\rightarrow$ “-1”.

- An LDPC code is defined by a sparse parity-check matrix H , where a valid codeword $\mathbf{c}$ must satisfy

$$H\mathbf{c}^T = 0 \pmod{2}.$$

If the message vector has length k and the codeword length is n , then the code rate is

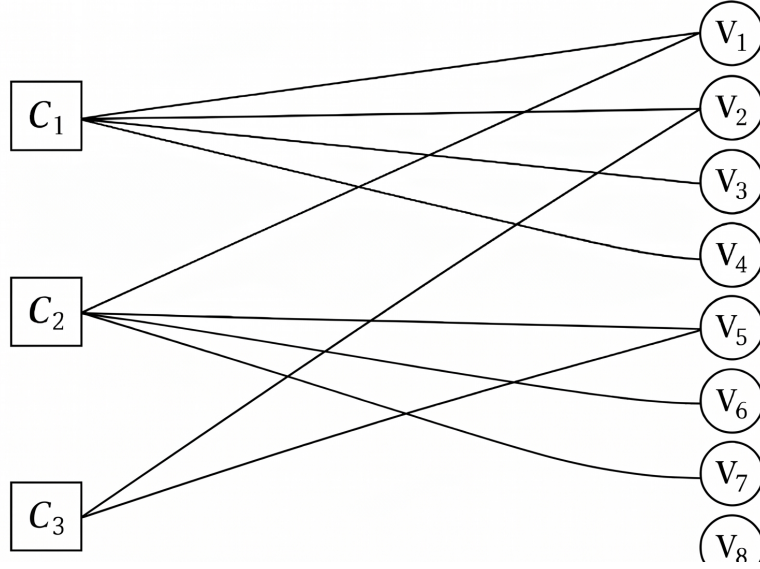
$$R = \frac{k}{n}.$$

The sparsity of H reduces decoding complexity and makes iterative decoding practical. In our implementation, the matrix is built from a small base matrix by replacing each entry with a circulant submatrix of size $Z \times Z$, where Z is the lifting factor. This produces a quasi-cyclic LDPC structure, which is efficient to store and process.

$$H = \begin{matrix} & V_1 & V_2 & V_3 & V_4 & V_5 & V_6 & V_7 & V_8 & V_9 & V_{10} \\ \begin{matrix} C_1 \\ C_2 \\ C_3 \\ C_4 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \end{matrix}$$

Figure 4: A general parity check matrix for an LDPC code

- In the parity-check matrix H , each row corresponds to a parity constraint and each column corresponds to a code bit. A value of 1 indicates that the corresponding variable node participates in that check equation, while 0 indicates no connection. Because the matrix is sparse, each check equation involves only a small number of bits, which is the key feature that enables efficient decoding. The structure shown in Figure below illustrates the Tanner-graph interpretation of the LDPC code, where variable nodes and check nodes are connected according to the nonzero entries of H .



- For encoding, the parity-check matrix is transformed into systematic form using Gauss–Jordan elimination over the binary field $\text{GF}(2)$:

$$H = [A \ I_{n-k}],$$

where A is a binary submatrix and I_{n-k} is the identity matrix. Once H is written in this form, the corresponding generator matrix is obtained as

$$G = [I_k \ A^T].$$

Here, I_k is the $k \times k$ identity matrix. This form allows the information bits to be encoded directly into a valid LDPC codeword. The code in our implementation performs this row reduction step and then constructs the transposed systematic matrix for encoding.

- To verify the validity of the encoded word, the syndrome vector is computed as

$$\mathbf{s} = H\mathbf{c}^T \pmod{2}.$$

If the encoding is correct, the syndrome must be the zero vector:

$$\mathbf{s} = \mathbf{0}.$$

A nonzero syndrome indicates that the codeword does not satisfy the parity constraints. In our implementation, this syndrome check is used to confirm that the generated codeword is a valid LDPC codeword.

6.3 LDPC Decoder

- LDPC decoding is the process of recovering the transmitted codeword from a noisy received signal using the parity-check matrix H . Due to channel noise, the received sequence may violate the parity constraints, so the decoder iteratively corrects errors by enforcing

$$H\mathbf{c}^T = 0 \pmod{2}.$$

This is achieved using soft information and message passing on the Tanner graph. The goal is to obtain the most probable valid codeword consistent with both the received signal and the code structure.

- The decoding algorithm is a weighted min-sum message-passing decoder. In each iteration, it exchanges messages between variable nodes and check nodes according to the Tanner graph defined by H . The check-node update uses the min-sum rule: the sign of the incoming messages is multiplied, and the minimum absolute value among the incoming messages is selected. In your implementation, this classical min-sum rule is modified by trainable weights, so the decoder learns how strongly to scale the messages at different stages

6.3.1 Min-Sum Iterative Decoding Algorithm

- The transmitted codeword $\mathbf{c}$ is mapped to BPSK symbols as

$$s_j = 1 - 2c_j,$$

so that $0 \mapsto +1$ and $1 \mapsto -1$. The received symbol is

$$y_j = s_j + n_j,$$

where n_j is additive white Gaussian noise (AWGN).

- In an ideal LLR-based decoder, the channel log-likelihood ratio is given by

$$L_j^{(0)} = \log \frac{P(c_j = 0 \mid y_j)}{P(c_j = 1 \mid y_j)} = \frac{2y_j}{\sigma^2}.$$

In the implementation, the noisy BPSK samples are used directly as the soft input, and the trainable weights learn the effective scaling during training. The training samples are generated by encoding random information bits, applying BPSK modulation, and then corrupting them with Gaussian noise at the chosen SNR.

- The parity-check matrix H is flattened and used to identify the connection pattern of the Tanner graph. A check node c_i is connected to a variable node v_j if

$$H_{ij} = 1.$$

For each check node c_i , define the set of neighboring variable nodes as $N(c_i)$. The decoder uses this sparse connectivity structure directly in the iterative message updates.

- LDPC decoding is an iterative belief-propagation process. Two types of messages are exchanged:

- Variable-to-check messages $L_{v \rightarrow c}^{(l)}$
- Check-to-variable messages $L_{c \rightarrow v}^{(l)}$

where l is the iteration index.

The fundamental idea is that:

1. each variable node collects information from the channel and from neighboring check nodes,
2. each check node enforces local parity constraints,
3. the process is repeated iteratively until convergence.

This allows the decoder to progressively improve the reliability of each bit estimate.

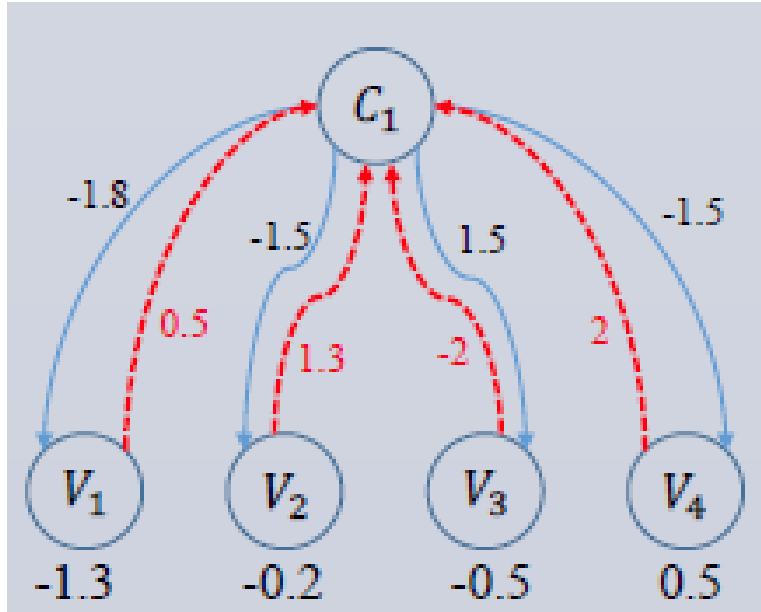


Figure 5: Variable-node update table and Tanner-graph message-passing diagram after the first iteration for check node C_1 .

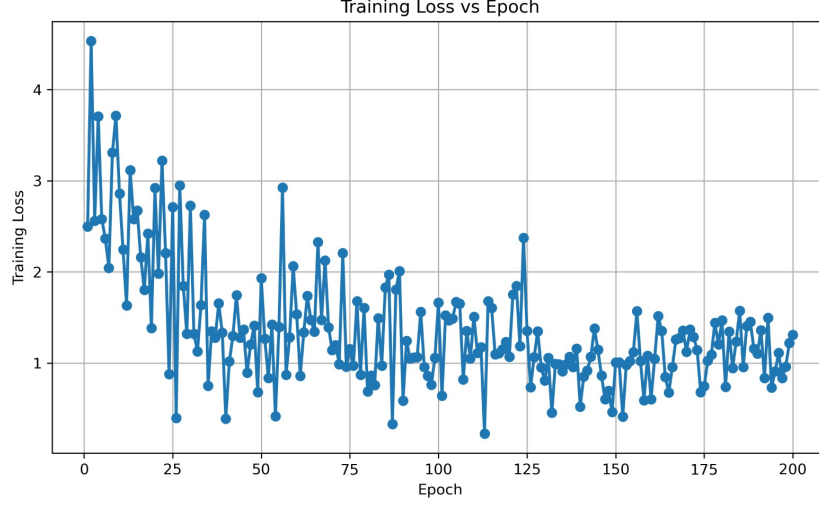
The Tanner-graph diagram in Figure 5 visualises the messages exchanged between check node C_1 and variable nodes V_1, V_2, V_3, V_4 , with the updated SUM row providing the soft-decision estimates used in subsequent iterations.

- For each check node c_i and each neighboring variable node $v_j \in N(c_i)$, the check-to-variable message in the weighted min-sum form is given by

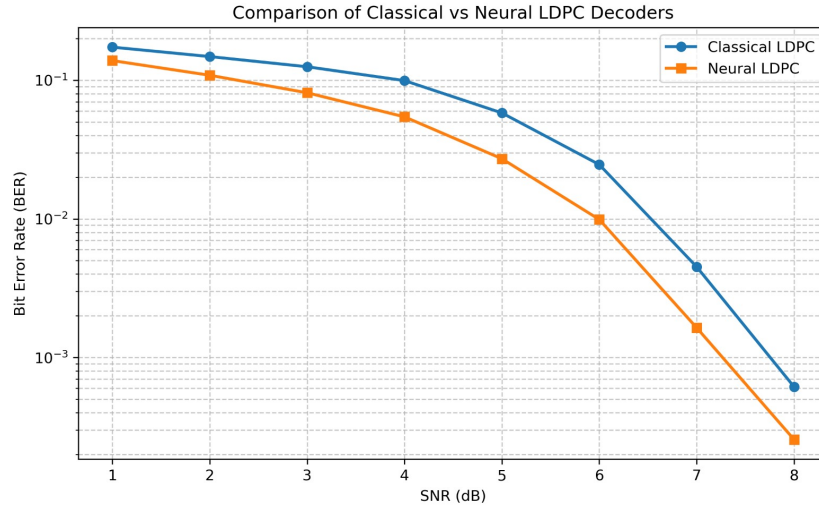
$$L_{c_i \rightarrow v_j}^{(l)} = \alpha_l \left(\prod_{v_k \in N(c_i) \setminus v_j} \text{sgn}(L_{v_k \rightarrow c_i}^{(l-1)}) \right) \min_{v_k \in N(c_i) \setminus v_j} |L_{v_k \rightarrow c_i}^{(l-1)}|.$$

where α_l is a learnable scaling factor for iteration l .

6.4 Result



- Figure shows the training loss of the neural LDPC decoder as a function of epoch. The loss decreases overall from a high initial value and then gradually stabilizes, indicating that the model is learning the decoding rule from the noisy training samples. Although short-term fluctuations are present due to stochastic mini-batch training, the downward trend confirms that the optimization process is effective. This suggests that the network is successfully adapting its parameters and moving toward a more reliable decoding performance.



- Figure compares the bit error rate (BER) performance of the classical LDPC decoder and the proposed neural LDPC decoder as a function of SNR. The BER decreases for both methods as SNR increases, which is expected because the channel becomes less noisy at higher SNR. However, the neural LDPC decoder consistently achieves lower BER than the classical decoder over the entire SNR range, indicating improved decoding accuracy and better robustness to channel noise.

7 Conclusions

This report has presented the complete, progressively enhanced development of a software-only CV-QKD simulation framework:

1. **GG02 Baseline:** We reproduced and validated the GG02 protocol, including state preparation, channel modelling, parameter estimation, LDPC error correction, and both asymptotic and composable key rate computation. Key rates decrease with increasing distance, excess noise, and decreasing SNR, as expected from theory.
2. **Post-Selection Enhancement:** By introducing Alice’s Gaussian probabilistic filter and Bob’s non-Gaussian notch filter, we demonstrated that purely software-driven post-selection can significantly boost key rates. Alice’s filter narrows the effective modulation variance to match the channel optimally; Bob’s filter removes low-SNR outcomes to emulate a higher-quality channel. The method achieves positive keys in regimes where unfiltered GG02 fails.
3. **Polar Code Error Correction:** Replacing LDPC with polar codes substantially reduced reconciliation latency while maintaining correction efficiency. BER increases from 0.000 to 0.104 and FER from 0.000 to 0.220 over 45 km, with the PA key length decreasing from 1024 to 798 bits — all consistent with the growing channel noise at longer distances.
4. **ML-Based Sifting:** The Isolation Forest sifting algorithm reliably detects and removes anomalous miniblocks that artificially inflate excess noise. Under high-noise or disturbed channel conditions, this yields significant SKR improvements. Under low-noise conditions, its performance remains comparable to the unsifted baseline, demonstrating robustness without degrading ideal-channel performance.
5. **Neural Network LDPC Codes:** The results show that neural-network-assisted LDPC decoding can learn effective error-correction strategies directly from noisy data. Compared to the classical LDPC decoder, the neural approach achieves lower BER across different SNR values, indicating improved decoding performance. This demonstrates that integrating machine learning with LDPC codes can enhance reconciliation efficiency in CV-QKD systems. Overall, it provides a practical, software-based improvement without requiring changes to the physical communication setup.

Together, these results demonstrate that combining polar codes with ML-based state filtering provides a computationally efficient, hardware-transparent pathway to practical high-performance CV-QKD.

8 Future Work

In future work, we plan to:

- Develop and train deep neural network architectures on large-scale CV-QKD data to learn complex noise patterns and decoding rules, exploring them as potential replacements or enhancements to polar-code SC decoding.
- Apply machine learning across all post-processing stages — parameter estimation, reconciliation, slicing/quantisation, information leakage estimation, and key-rate optimisation — to study how data-driven methods impact overall system performance.

- Explore integration of trained models with polar codes to improve reconciliation efficiency and further raise secret key rates without hardware changes.
- Extend the framework to satellite and free-space optical channels, where rapidly fluctuating channel conditions make adaptive software-level processing especially valuable.

Acknowledgements

We would like to thank Prof. Neel Kanth Kundu and Prof. Bhaskar Kanseri for their invaluable guidance and support throughout this project. We also acknowledge the Department of Physics, IIT Delhi, for providing infrastructure and computational resources. Special thanks to our peers and colleagues for helpful discussions and feedback during the development of this work.

References

- [1] A. G. Mountogiannakis, P. Papanastasiou, B. Braverman, and S. Pirandola, “Composable secure data processing for Gaussian-modulated continuous variable quantum key distribution,” *Physical Review Research*, 2022.
- [2] O. Erkilic, B. Shajilal, and L. O. Corlan, “Enhanced continuous-variable quantum key distribution protocol via adaptive signal processing,” arXiv preprint, July 2025.
- [3] J. Xie, L. Zhang, Y. Wang, and D. Huang, “Deep neural network based reconciliation for CV-QKD,” *Photonics*, vol. 9, p. 110, 2022.
- [4] D. Jin, Y. Guo, Y. Wang, Y. Li, and D. Huang, “Key-sifting algorithms for continuous-variable quantum key distribution,” 2020.
- [5] C. Weedbrook, S. Pirandola, R. García-Patrón, N. J. Cerf, T. C. Ralph, J. H. Shapiro, and S. Lloyd, “Gaussian quantum information,” *Reviews of Modern Physics*, vol. 84, pp. 621–669, 2012.
- [6] X. Liu, N. Wang, C. Xu, S. X. Ng, P. V. Trinh, S. Sugiura, and L. Hanzo, “Tetherless THz CV-QKD — Would it work?” *IEEE Wireless Communications Magazine*, 2022.
- [7] **CV-QKD Simulator Repository** – <https://github.com/liplahm/cvqkd-simulator>